

CODEBASE REVIEW — EXPLAINER

RFP Sentinel

A buyer-side, multi-agent RAG platform for GeM Electronics procurement: checks tenders against government rules before publication, and checks bids against tender requirements after submission — with a human confirming every consequential decision.

Generated directly from the current source tree — every claim below is traceable to a real file and function. Where the codebase doesn't (yet) implement something, this document says so explicitly rather than describing an idealized design.

Table of Contents

1. Executive Summary	3
2. Technology & Tooling Choices	4
3. Full Application Architecture	8
4. Why Multi-Agent (and Why Not a Fixed Pipeline)	10
5. Knowledge Base & Norm Checking (Phase 1)	12
6. RFP Ingestion & Human Checkpoint A	14
7. Bidder Documents: Storage, Linking & Search	16
8. Evaluation Flow: Stage 1 & Stage 2 (Phase 2)	19
9. End-to-End Walkthrough	21
10. Complete API Reference	23
11. Glossary	26
12. Anticipated Review Questions & Answers	28
Appendix — Codebase Inventory	30

1. Executive Summary

RFP Sentinel is software that helps a government buyer run a tender fairly. A buyer uploads a tender (RFP) document; the system automatically checks its own eligibility and technical clauses against real government procurement rules — flagging anything that looks non-compliant, with a citation to the exact rule, before the tender is published. Once companies (bidders) submit their bids, the system checks each one against the tender's own requirements and produces a ranked, evidence-backed result — but a human always confirms the consequential steps; nothing is auto-published and no winner is auto-selected.

The one-line positioning: almost every AI tool in government-tender space is built for the company *bidding* on a tender, helping it write a better response. RFP Sentinel is built for the other side — the buyer / Technical Evaluator who has to check the tender is fair and then evaluate what comes back, honestly and defensibly. That is the load-bearing differentiator of this whole project, not an incidental feature.

Who uses it

- **Buyer (Technical Evaluator role):** uploads the RFP, reviews anything flagged at Checkpoint A, publishes, later closes bidding and walks through Stage 1/Stage 2 evaluation.
- **Bidder:** browses published tenders without logging in, sees a plain-language summary and a legitimacy check, and — once signed in — submits a bid (sealed technical + financial packets).
- **Admin:** manages the norm knowledge base's active/superseded/withdrawn status, suspends or restores user accounts, and audits any tender that was published despite an unresolved flag.

Note: the prompt that requested this document also names a "Competent Authority" and "Bid Evaluation Committee under GFR 2017 Rule 189" as distinct users. The codebase implements exactly three roles — `buyer`, `bidder`, `admin` (see `backend/db.py` users table CHECK constraint) — with no separate Competent Authority / BEC role or approval step. GFR Rule 189's two-envelope (sealed bid) *principle* is genuinely implemented (see §7 and §8); a distinct Competent Authority persona is not.

What problem it actually solves

On India's Government e-Marketplace (GeM), a buyer department publishes a tender, companies submit bids, and someone has to fairly compare every bid against the rules and pick a winner — manual, slow, and easy to get wrong or successfully challenge later. RFP Sentinel automates the mechanical, checkable parts of that comparison (does this clause conflict with a known rule? does this bid's document actually address this requirement?) while keeping every consequential decision — publishing a flagged tender, resolving an ambiguous bid, breaking a price tie — in human hands, with the reasoning recorded.

2. Technology & Tooling Choices

Ollama — local/self-hosted LLM & embedding server

What it is	A program that runs open-source AI language models on your own computer or server, instead of calling a company's cloud API. RFP Sentinel currently points at a remote Ollama instance (<code>OLLAMA_BASE_URL</code>) running two models: <code>qwen2.5:7b</code> for reasoning/classification and <code>bge-m3</code> for embeddings (see below).
Why chosen	Government tender/bid data is sensitive; sending it to a third-party cloud LLM is a real compliance concern most buyers wouldn't accept. Ollama keeps every document, clause, and verdict on infrastructure the project controls.
Advantages	No per-token cost, no data leaving the org's network, works fully offline once models are pulled, simple HTTP API (<code>/api/generate</code> , <code>/api/embeddings</code>).
Disadvantages	Local/open models are meaningfully weaker at nuanced reasoning than frontier cloud models (see the Llama 3.2 3B finding below); CPU-only inference is slow (each classification call takes several seconds); no built-in high availability.
Alternatives rejected	Cloud LLM APIs (OpenAI, Anthropic, etc.) — rejected on data-residency/cost grounds for a government-procurement use case, not capability.

A real, documented finding that shaped this project's architecture: the original model tried was `llama3.2:3b` . Testing found it reliably *wrong* on numeric threshold comparisons — given a clause requiring "minimum 10% discount" against a bid offering 12%, the model correctly stated $12 > 10$, then still concluded "fail" — reproducibly, even at temperature 0, across prompt rewrites. A deterministic regex-based threshold checker (`threshold_check.py`) was built specifically to route numeric comparisons around the LLM entirely. Once the project switched to the larger `qwen2.5:7b` (currently in use, run on a separate, more capable machine) and confirmed via a direct side-by-side rerun that it got the same comparisons right, `threshold_check.py` was deleted outright — it is no longer part of the codebase. This is genuinely useful review material: it shows the team caught a real reasoning failure empirically, built a safety net, and then removed the safety net only after re-verifying it was no longer needed — not by assumption.

bge-m3 — embedding model

What it is	An embedding model turns text into a list of numbers (a <i>vector</i>) that captures its meaning, so that similar meanings end up as nearby numbers — this is what makes semantic search possible. <code>bge-m3</code> produces 1024-number vectors.
Why chosen	Upgraded from the originally-used <code>nomic-embed-text</code> (768-dim) for stronger retrieval quality on dense procurement/legal text. This is a very recent change — the entire <code>norms</code> Qdrant collection was deleted and rebuilt from scratch at the new 1024 dimension.
Advantages	Stronger semantic matching on long, legal-style text than the smaller model it replaced.
Disadvantages	Larger model, slower per-call on CPU-only hardware; any future embedding-model change again requires a full collection rebuild and re-ingestion (vector dimension is fixed per Qdrant collection).
How chunks are actually sent	Read the code directly (backend/rag/embeddings.py): <code>embed_texts()</code> calls <code>embed_text()</code> in a plain Python loop — one HTTP call per chunk, strictly sequential, never

batched. The code comment is explicit about why: *"no GPU here, so keeping memory use predictable matters more than the speed a batched call might give."*

Qdrant — vector database

What it is	A database purpose-built to store vectors and quickly find the "nearest" ones to a query vector — the engine behind semantic search.
Why chosen	Norm documents change over time (a policy gets amended); Qdrant's payload-filtering lets old and new versions of a norm coexist as ordinary rows, distinguished by a <code>status</code> field, with no need to delete/rebuild on every update.
Advantages	Fast approximate-nearest-neighbor search; payload filtering (e.g. <code>status=active</code>) baked directly into the query, not applied after the fact in application code.
Disadvantages	A second database to run and back up alongside Postgres; not relational — no foreign keys or joins.
Alternatives rejected	Storing embeddings as a column inside Postgres (e.g. via pgvector) is not used in this codebase — two purpose-specific stores were chosen instead (see the Qdrant-vs-Postgres discussion below).

PostgreSQL

What it is	A relational (table-based) database.
Why chosen	Two genuinely different jobs need it: (1) real, queryable metadata — users, published tenders, submitted bids, and their FK relationships (<code>backend/db.py</code>); (2) LangGraph's own checkpoint storage, via the <code>langgraph-checkpoint-postgres</code> package.
Advantages	ACID transactions, foreign-key integrity (e.g. a bid can't exist without its RFP), a single connection pool serves both uses.
Disadvantages	Not suited to vector similarity search at any real scale — this is exactly why Qdrant exists alongside it, not instead of it.

Qdrant vs. Postgres — why two stores, not one

They store fundamentally different kinds of things. Postgres holds facts with clear structure and relationships you'd write SQL against ("which bids belong to RFP X," "is this user active"). Qdrant holds document chunks as vectors you search by *meaning*, not by exact match — "which norm clauses are conceptually related to this criterion" has no meaningful SQL equivalent. The same LangGraph checkpointer and the real metadata tables share one Postgres connection pool (see `backend/main.py`'s `lifespan()`) — deliberately one pool, two uses, not two separate connections to manage.

LangGraph — orchestration framework

What it is	A library for building multi-step AI workflows as an explicit graph of steps ("nodes") with shared state that can pause and resume — including pausing for a human to look at something before continuing.
Why chosen	The RFP-intake pipeline (extract → check compliance → check prohibited practices → human review) needs to run several expensive LLM steps, then <i>durably pause</i> for a human, possibly for hours or days, then resume exactly where it left off. See §4 for the detailed "why not a fixed script" argument.

Advantages	The pause/resume ("interrupt") is persisted to Postgres automatically — restart the whole backend process and a paused evaluation is still exactly where it was.
Disadvantages	Only the RFP-intake phase is actually built as a LangGraph graph today (see §4) — Stage 1/Stage 2 bid evaluation run as plain background functions instead, not as further graph nodes.

LangGraph vs. LlamaIndex — the prompt asked for this framed as different layers, and that framing is correct in general, but: LlamaIndex is **not used anywhere in this codebase**. There is no reference to it in any file. Retrieval here is a small, hand-written layer (`backend/rag/qdrant_client.py`, `backend/rag/embeddings.py`) that calls Qdrant and Ollama directly. The conceptual point still stands and is worth stating in review — LangGraph is the orchestration layer (state, routing, pausing), retrieval is a separate concern — this project just implements that retrieval layer itself rather than adopting LlamaIndex to do it.

FastAPI

What it is	A Python web framework for building APIs.
Why chosen	Async-friendly, automatic request validation via Pydantic models (the same library used for all of this project's own data shapes), minimal boilerplate.
Advantages	Background tasks (<code>BackgroundTasks</code>) run slow LLM-heavy work off the request thread without needing a separate task queue.
Disadvantages	No built-in task queue/retry semantics — a crashed background task is only as recoverable as the code makes it (see §4's discussion of failure handling).

Streamlit is NOT part of the current stack. An earlier prototype used Streamlit for the frontend; it has since been fully replaced by a React + Vite + Tailwind CSS single-page app, and the legacy Streamlit code was deleted from the repository entirely in this same working session. If your prompt or slide deck still says "Streamlit app," that is now out of date — say "the React frontend" in review.

RAG over fine-tuning — why

RAG (Retrieval-Augmented Generation) means: instead of baking facts into a model's weights via fine-tuning, keep the facts in an external, searchable store and hand the model the relevant passages at question time. Two independent reasons this project uses RAG, not fine-tuning:

- **Norm versioning.** Government policy changes (a percentage target amended, an order superseded). Qdrant's `status` field flips a norm from `active` to `superseded` instantly (`mark_status()` , `backend/rag/qdrant_client.py`) — no retraining, no downtime. Fine-tuning a model on old rules would require retraining every time a policy changes, with no clean way to "roll back" to the old rule's exact text for an audit.
- **Hardware.** This project runs on CPU-only hardware with no dedicated GPU (confirmed directly on the development machine — no NVIDIA driver present). Fine-tuning even a small open model at reasonable quality needs GPU compute this project's target environment doesn't have; RAG needs only inference, which — while slow — is at least achievable on CPU.

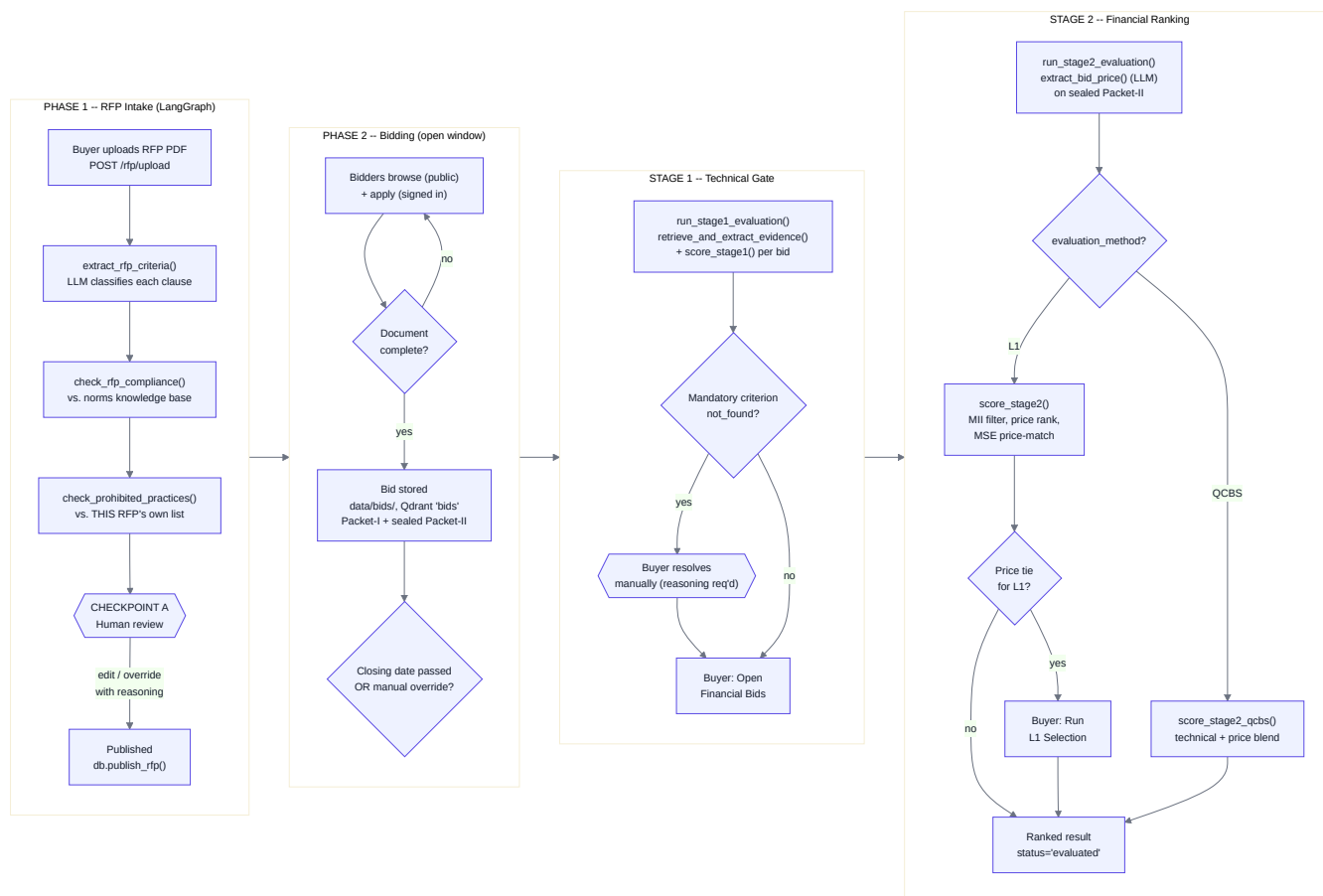
Deterministic weighted scoring over LLM-generated scores

The LLM's job stops at a categorical verdict per criterion (`pass` / `fail` / `partial` / `not_found`). Turning those verdicts into a pass/fail gate and a ranked list is pure, unit-tested Python (`backend/scoring/scoring.py`) with **zero LLM or Qdrant dependency** — the module docstring says this explicitly, and `tests/test_scoring.py` exercises it against hand-built fixtures, not a live model.

This is an auditability decision, not a capability workaround. The reason isn't "the LLM can't do arithmetic" (though the Llama 3.2 3B finding above shows it sometimes genuinely can't reason correctly either) — it's that a government tender outcome must be explainable and reproducible on demand. "Bid A beat Bid B because `technical_score` = $\Sigma(\text{weight} \times \text{verdict_score}) / \Sigma(\text{weight})$, computed by this exact formula" is defensible in a way "the model said so" never can be, regardless of how good the model gets.

3. Full Application Architecture

Four layers, each with a narrow job. Requests flow strictly downward; only the layer that owns a piece of data touches its store.



The full upload-to-result flow, across every layer — every checkpoint and decision point labelled.

Layer by layer

Layer	Technology	What crosses its boundary
Frontend	React 19 + Vite + Tailwind CSS v4, React Router, Axios (frontend/src/)	JSON over HTTPS to FastAPI; a JWT bearer token on every authenticated call
API	FastAPI (backend/api/*.py, backend/main.py)	Validates requests (Pydantic), checks JWT + role + <code>is_active</code> , either answers directly from Postgres/Qdrant or hands off to the graph/background tasks
Orchestration	LangGraph, compiled once at startup (backend/graph/build_graph.py)	Structured state in, structured state out; pauses execution at <code>checkpoint_a</code> and persists to Postgres
Reasoning / storage	Ollama (LLM + embeddings), Qdrant, Postgres	Prompts and embedding requests to Ollama; vector queries to Qdrant; SQL to Postgres

What's genuinely NOT in this architecture

- No message queue / task queue (Celery, RabbitMQ, etc.) — background work uses FastAPI's own `BackgroundTasks`, which runs a plain sync function in a thread pool.
- No separate microservices — one FastAPI process serves every role's endpoints, gated by dependency-injected role checks, not separate deployments.
- No caching layer (Redis, etc.) beyond one specific case: a generated RFP summary is cached as a column in Postgres (`rfps.summary`) so it's computed once, not on every page view.

4. Why Multi-Agent (and Why NOT a Fixed Pipeline)

Every agent, and the specific judgment call it makes

Agent (file → function)	The decision it makes	Why it needs judgment, not a rule
<code>extract_rfp_criteria.py → _is_guidance_text()</code>	Is this clause a real requirement, or boilerplate instructional text aimed at the buyer?	No fixed phrase list reliably distinguishes these across different RFP templates — it's a reading-comprehension judgment.
<code>extract_rfp_criteria.py → _infer_mandatory()</code>	Is this criterion a strict "must," or a preference?	Depends on the clause's own wording and framing, not a keyword.
<code>extract_rfp_criteria.py → _infer_category()</code>	Classify as technical / financial / eligibility / other	Requires understanding what the clause is actually regulating, including an honest "other" for clauses that fit none of the three.
<code>check_rfp_compliance.py → check_rfp_compliance()</code>	Does this criterion conflict with a retrieved government-norm clause?	"Conflict" here means semantic contradiction, not string overlap — the same module's docstring notes the earlier small model got numeric comparisons backwards even while correctly identifying the numbers.
<code>check_prohibited_practices.py → check_prohibited_practices()</code>	Does this criterion actually perform one of GeM's own listed buyer drafting-mistakes?	Must distinguish "matches the same topic" from "actually does the prohibited thing" — a genuine judgment call, not a keyword match.
<code>retrieve_and_extract_evidence.py</code>	Does this bidder's document content satisfy this specific criterion?	Free-form document text rarely restates a requirement verbatim; deciding "satisfied / not satisfied / partially" requires reading comprehension.
<code>extract_bid_price.py → extract_bid_price()</code>	Which figure in a free-form price schedule is the real total?	Documents list subtotals, per-unit rates, and tax lines; picking "the" total requires understanding document structure, not a regex.
<code>generate_rfp_summary.py</code>	Write a plain-language summary of what's being procured and how it'll be judged	Synthesis/writing, not extraction.

Explicitly NOT agents, despite living next to the ones above: `check_document_completeness()` (normalized filename substring matching) and `check_rfp_legitimacy()` (fixed keyword-to-norm-name lookup) make zero LLM calls. Both modules' own docstrings are explicit that this is deliberate — a fast, honest, deterministic check where an LLM would add cost and non-determinism for no real benefit.

The reviewer's likely challenge: "Couldn't this just be plain Python?"

Each individual classification call in the table above genuinely could be one Python function with one LLM call inside it — and several of them are exactly that. What plain, linear Python *cannot* give you, and what LangGraph specifically provides here:

- **Stateful, durable pausing.** `checkpoint_a` calls LangGraph's `interrupt()` (`backend/graph/build_graph.py`), which pauses execution and persists the entire accumulated state to Postgres via `PostgresSaver`. A buyer can come back hours later, on a different request, possibly after the backend process itself has restarted, and `graph.invoke(Command(resume=...))` picks up exactly where it left off. A plain function call has no equivalent — its state lives in memory and dies with the process or the request.
- **Explicit, inspectable state.** The shared `EvaluationState` (`backend/graph/state.py`) is a typed object every node reads and returns updates to — `graph.get_state(config)` can be queried by any endpoint at any time to answer "what's this RFP's status right now," which is exactly how `GET /rfp/{id}/status` works.
- **A single compiled graph, reused across every RFP.** `build_graph()` is called once at FastAPI startup (`backend/main.py`'s `lifespan()`); each RFP is just a different `thread_id` against the same compiled graph, not a new pipeline instance.

What LangGraph itself is doing vs. what the agents do, precisely: LangGraph owns state-passing between nodes, the fixed edge sequence (`extract` → `compliance` → `prohibited-practices` → `checkpoint_a` → `END`), and the interrupt/resume mechanism. Each node function owns its own actual work — parsing a PDF, calling an LLM, running a deterministic check — and returns a plain dict of state updates. LangGraph never inspects *what* a node decided; it only knows a node started, finished, and returned some state.

Honest limitation, worth pre-empting in review: LangGraph checkpoints *between* nodes, not mid-node. If a node raises an exception partway through its own internal loop (e.g. `check_rfp_compliance` failing on criterion 20 of 22 because a remote network call died), nothing about that partial progress is saved — the whole node must re-run from its start on retry. The current code (`backend/api/rfp.py` → `_run_to_checkpoint_a()`) catches such an exception and writes an explicit `status="failed"` into the graph state so the failure is visible rather than silent, but there is no finer-grained, mid-node resume.

Cycles/retries in the current graph

To be precise rather than aspirational: the compiled graph itself (`backend/graph/build_graph.py`) is a straight line — `extract_rfp_criteria` → `check_rfp_compliance` → `check_prohibited_practices` → `checkpoint_a` → `END` — with no conditional branch and no cycle back to an earlier node. The "retry" behavior that exists in this system lives one level lower, inside the LLM-calling functions themselves (`classify()` in `backend/llm/ollama_client.py` retries a malformed/network-failed LLM call up to 3 times), not as a LangGraph-level conditional edge.

5. Knowledge Base & Norm Checking (Phase 1)

The norm documents actually in the knowledge base

The prompt's assumed list of six ("GeM GTC, GFR 2017, DPIIT Make-in-India Order, MeitY Electronics CRS Order, BIS CRS list, MSME Procurement Policy") does not match `data/norms/manifest.json`. The real, currently-ingested list is:

Norm	Version
GeM General Terms and Conditions (GTC)	4.0.1.23
MeitY CRS Applicant Handbook	unversioned in source
MSME Public Procurement Policy Order	2012
GFR 2017, Chapter 6 (Procurement of Goods and Services only — pages 40–56 of a 208-page document)	2017
DPIIT Public Procurement (Preference to Make in India) Order 2017	2020-06-04 revision
DPIIT Public Procurement (Preference to Make in India) Order 2017 (amendment)	2020-09-15 revision (amends specific paragraphs of the one above; both stay active simultaneously)

There is no separate "BIS CRS list" document, and the CRS Handbook is the MeitY CRS document — the prompt's six-item list appears to describe an earlier planning stage, not the current, real manifest. A seventh PDF (an Office Memorandum on concurrent MSME/Make-in-India policy application) sits in `data/norms/` but is **not yet in the manifest and not yet ingested** — it is a fully scanned, image-only PDF (confirmed directly: 4 pages, zero extractable text on any page), and this project's text extraction has no OCR fallback yet (see the gap noted below).

Building the knowledge base, step by step (read directly from the code)

1. **Extract text** — `ingestion/extract_text.py` → `extract_text_by_page()`, via `pdfplumber`, one page at a time. Automatically detects and retries pages that show a "doubled letter" rendering artifact from certain PDFs' fake-bold text encoding.
2. **Extract tables** — `ingestion/extract_tables.py`, same `pdfplumber` pass, serialized to pipe-delimited rows so column structure survives into the embedding.
3. **Language filter** — `ingestion/language_filter.py` → `filter_english()`, strips Devanagari (Hindi) text before chunking, so a chunk boundary never lands mid-stripped-paragraph.
4. **Chunk** — `ingestion/chunker.py` → `chunk_document()`. *Chunking* means splitting a long document into smaller pieces small enough to embed and retrieve individually. **This is section-aware, not a fixed sliding window**: a chunk never spans two different clauses. Read directly from the code's own constants:
 - A detected section (bounded by a numbered clause, lettered sub-item, or an all-caps heading) becomes one chunk whole if it's ≤ `MAX_SECTION_WORDS = 1500` words.
 - An oversized section is sub-split into `TARGET_WORDS = 1200` -word windows, with `OVERLAP_WORDS = 180` (≈15%) of overlap between consecutive sub-chunks — but this overlap never crosses into the next section's content.
 - Tables are never split by word count — one table is one chunk, or (if over 20 rows) split into row-groups of 20 with the header repeated in each group.

5. **Embed** — `backend/rag/embeddings.py` → `embed_texts()`. As stated in §2: **one Ollama HTTP call per chunk, sequential, not batched.**
6. **Store** — `backend/rag/qdrant_client.py` → `upsert_chunks()`. Each chunk becomes one Qdrant point in the `norms` collection, with a **deterministic UUID5 point ID** derived from `doc_id` + chunk index — re-running ingestion after a bug fix overwrites the same points instead of creating duplicates.

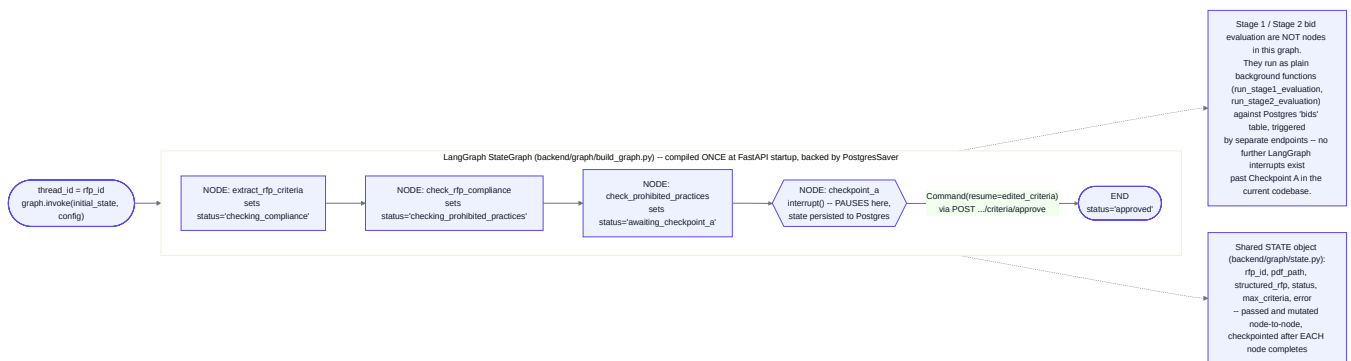
Where norms physically "sit," and how a check actually runs

Every chunk's Qdrant payload carries: `doc_id`, `norm_name`, `status`, `version`, `effective_date`, `clause_ref`, `chunk_type` (`prose/table`), `page_number`, `text`. A compliance check (`check_rfp_compliance()`) runs as: embed the criterion's text → `search_active()` queries Qdrant for the top-5 nearest norm chunks, **with a hard filter `status = active` baked into the query itself** (not applied afterward in Python) → the retrieved chunks plus the criterion go to `classify()` for a compliant/violation/unclear verdict with a citation.

Norm versioning

Superseding a norm never deletes anything. `mark_status()` (`backend/rag/qdrant_client.py`) flips every chunk belonging to a `norm_name` to `superseded` or `withdrawn` via a payload-only update — the vectors and text stay in Qdrant, just excluded from `search_active()`'s results going forward. This is exposed to an admin via `POST /admin/norms/{norm_name}/status`.

6. RFP Ingestion & Human Checkpoint A



The compiled LangGraph graph itself — four nodes, one interrupt, backed by Postgres. This is the orchestration core discussed conceptually in §4.

How an RFP enters the system

A buyer uploads an actual RFP/tender PDF via `POST /rfp/upload` — not a structured web form. This is a deliberate, explicit design choice: the prompt notes that GeM's own RFP *creation* process is a structured, UI-driven flow (category selectors, eligibility filters, an ATC module, an evaluation-method picker). RFP Sentinel does not reimplement that authoring UI — it ingests the *finished PDF output* of that process (the same kind of document publicly listed on bidplus.gem.gov.in) and extracts structure back out of it via `extract_rfp_criteria()`.

Separating technical from financial/eligibility criteria — the actual mechanism

Each `Criterion` object (backend/models/rfp.py) carries a `category` field, set by the `_infer_category()` LLM classifier (see §4's table) to exactly one of: `technical`, `financial`, `eligibility`, or an honest `other` for clauses that fit none of the first three (e.g. delivery logistics, quantity-split rules). This category field is not cosmetic — `scoring.py`'s `_SCORED_CATEGORIES = {"technical", "financial", "other"}` constant means **eligibility criteria gate Stage 1 as a binary pass/fail but never contribute to the weighted technical_score** — a deliberate scoring-design decision, not an oversight.

Separately, each criterion carries a `mandatory` boolean (also LLM-classified) — this is what actually decides Stage 1's binary gate (any mandatory criterion that fails disqualifies the bid), independent of the technical/financial/eligibility split.

Checkpoint A — why it's a compliance requirement, not a UX nicety

Once all three Phase-1 checks (extraction, norm-compliance, prohibited-practices) complete, the graph reaches `checkpoint_a`, which calls LangGraph's `interrupt()` — this is a genuine pause, not a loading spinner: the graph's execution state is written to Postgres and the process returns control entirely. The buyer reviews every criterion, with any flagged one shown inline with its citation; anything flagged for a compliance or prohibited-practice issue **requires a written override reasoning before it can be published anyway** — enforced twice: once client-side in the review UI, and again server-side in `POST /rfp/{id}/criteria/approve` (backend/api/rfp.py), so a direct API call can't silently bypass the requirement. Approving resumes the graph via `graph.invoke(Command(resume=edited_criteria), config)`, and the RFP is published into the real Postgres `rfps` table at that exact moment (`publish_rfp()`).

The prompt frames Checkpoint A as required by "the two-stage double-packet evaluation" specifically. More precisely: the double-packet (sealed Packet-I/Packet-II) mechanism is a *Stage 1/Stage 2 bid-evaluation* safeguard (see §8), separate from Checkpoint A, which is an RFP-*drafting*-stage human review. Both exist and both matter, but they are not the same mechanism — worth keeping distinct in review.

7. Bidder Documents: Storage, Linking & Search

Bidder documents are evaluator/bidder-UPLOADED, never fetched from GeM. Real GeM bid submissions are confidential to the buyer and the bidder — there is no public API to pull a third party's submitted documents, and this codebase makes no attempt to. Every bid document in this system arrived via an explicit upload — either through `POST /bidder/bids/{rfp_id}/submit` (a signed-in bidder, the real production path) or the standalone `ingestion/ingest_bid.py` CLI script (manual/test ingestion).

Where files physically live, and how multiple documents per bidder are handled

On submission (`backend/api/bidder.py` → `submit_bid()`), a fresh `bid_id` is generated and every uploaded file is written to disk at `data/bids/{bid_id}/{original_filename}`. The bidder submits a *list* of technical/eligibility files (Packet-I, can be several PDFs) plus exactly one financial document (Packet-II).

Read directly from `ingestion/ingest_bid.py`: every file for a `bid_id` goes through the identical extract → filter → chunk → embed pipeline used for norms, and all resulting chunks are pooled into the same Qdrant `bids` collection, tagged per-point with `bid_id`, `rfp_id`, `bidder_name`, `source_file`, `packet` ("I" or "II"). Multiple Packet-I files from the same bidder are *not* kept as separate logical documents at query time — they're distinguished only by the `source_file` payload field, and a search across `packet="I"` for a given `bid_id` can return chunks from any of that bidder's technical files, pooled together.

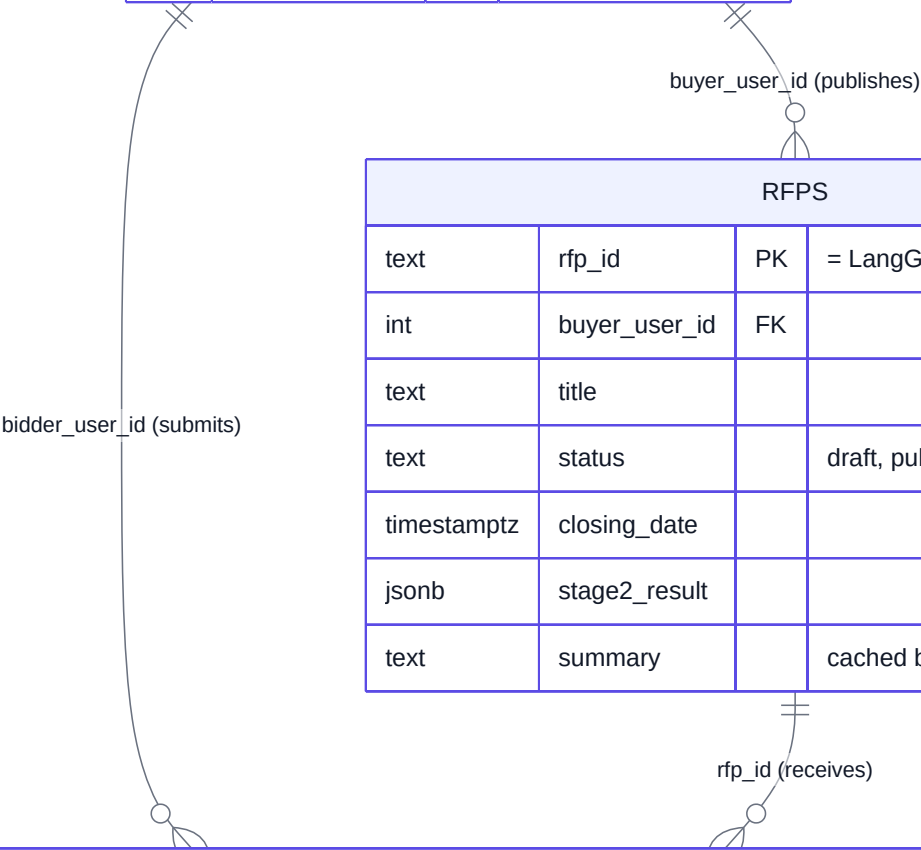
The sealed-packet mechanism — enforced structurally, not by convention

`search_bid()` (`backend/rag/qdrant_client.py`) **defaults its `packet` parameter to "I"**. Every caller that checks technical evidence (`retrieve_and_extract_evidence.py`) never passes `packet="II"` at all — the only code path that ever reads Packet-II is `extract_bid_price()`, called exclusively from `run_stage2_evaluation()`, which itself only runs after the buyer's explicit "Open Financial Bids" action, and only for bids that already passed Stage 1. A technically disqualified bidder's price is therefore never even queried, let alone read — this mirrors GFR Rule 189's real two-envelope principle at the code level, not just in the UI copy.

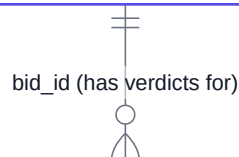
USERS			
int	id	PK	
text	email	UK	
text	password_hash		
text	role		buyer, bidder, admin
text	org_name		
bool	is_mse		
bool	is_mii_local		
bool	is_active		session revocation flag

RFPS			
text	rfp_id	PK	= LangGraph thread_id
int	buyer_user_id	FK	
text	title		
text	status		draft, published, closed, evaluated
timestampz	closing_date		
jsonb	stage2_result		
text	summary		cached bidder summary

BIDS			
text	bid_id	PK	
text	rfp_id	FK	
int	bidder_user_id	FK	
text	status		submitted, under_evaluation, stage1_passed, stage1_failed
numeric	price		NULL until Packet-II opened at Stage 2
numeric	technical_score		



jsonb	failed_criteria		
jsonb	pending_criteria		



BID_EVIDENCE			
int	id	PK	
text	bid_id	FK	
text	criterion_id		
text	verdict		pass, fail, partial, not_found
text	resolved_verdict		human override, if any
jsonb	citation		

The real Postgres foreign-key relationships — every one confirmed directly from backend/db.py's schema.

How "which bid belongs to which RFP / which user" is actually answered

Not via Qdrant — via ordinary Postgres foreign keys. `bids.rfp_id` references `rfps.rfp_id`; `bids.bidder_user_id` references `users.id`; a unique index `bids_rfp_bidder_unique ON bids(rfp_id, bidder_user_id)` enforces one bid per bidder per RFP (no amend/resubmit flow exists yet). Qdrant's `bid_id / rfp_id` payload fields are a parallel tag for isolating *content* search, not the system of record for the relationship itself.

How the public "search bids" feature actually works

`GET /bids` (backend/api/bids.py → `browse_bids()`, no login required) is a plain Postgres query — `list_published_rfps()` in backend/db.py — filterable by `keyword` (matched via SQL `ILIKE` against the title or the real GeM bid number), `category`, and `status`. This is ordinary relational search, not a Qdrant semantic query — semantic/vector search in this system is reserved for the internal compliance and evidence-matching steps, not the public browse listing.

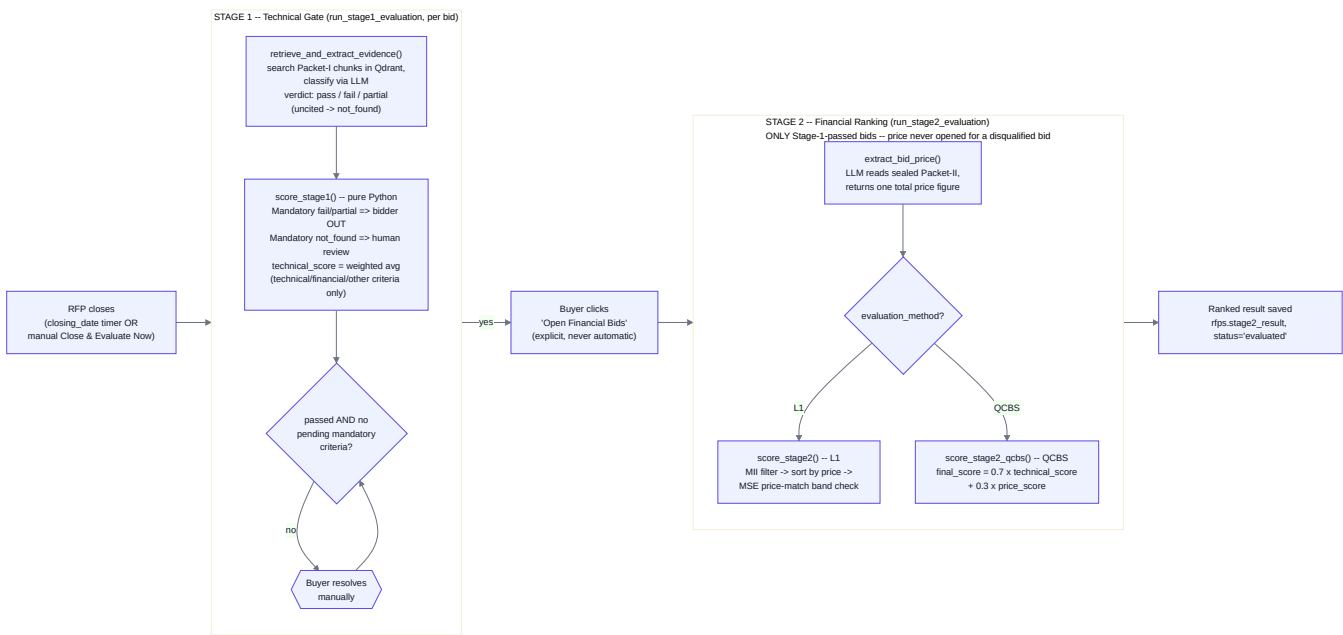
8. Evaluation Flow: Stage 1 & Stage 2 (Phase 2)

What triggers evaluation

Two paths, same function (`run_stage1_evaluation()`): (1) a background 60-second poll loop in `backend/main.py` (`_closing_timer_loop()`) that finds published RFPs whose `closing_date` has passed and closes+evaluates them automatically; (2) the buyer's manual `POST /rfp/{id}/close` ("Close & Evaluate Now" in the UI) — a demo-friendly override so a reviewer doesn't have to wait out a real multi-day bid window. Both paths converge on the identical evaluation code.

The real sequence, chunk by chunk (not assumed — read from the code)

Chunking and embedding for a bid's documents happen once, at *submission* time (`ingestion/ingest_bid.py`), not at evaluation time. By the time Stage 1 runs, every bid's chunks are already sitting in Qdrant. Stage 1 evaluation itself does not re-chunk or re-embed anything — for each buyer-approved criterion, it (a) embeds the *criterion's* text, (b) searches the bid's already-stored Packet-I chunks for the top-5 nearest matches, (c) sends the criterion plus those matches to the LLM classifier. This repeats once per criterion, per bid — sequential, not batched, same as ingestion.



Stage 1 (technical gate) vs. Stage 2 (financial ranking) — every decision point labelled.

Stage 1 filtering — basis, agent, and decision

Step	What decides it
Per-criterion verdict	Agent: <code>retrieve_and_extract_evidence()</code> 's LLM classifier — pass / fail / partial (an uncited verdict is downgraded to <code>not_found</code> rather than trusted).
Pass/fail gate	Not an agent — plain Python: <code>score_stage1()</code> . Any mandatory criterion verdict of <code>fail</code> or <code>partial</code> disqualifies the bid outright (no partial credit on a mandatory "must"). A mandatory <code>not_found</code> is held for human review, never auto-failed.

Technical score	Weighted average of verdict scores (<code>pass=1.0</code> , <code>partial=0.5</code> , <code>fail/not_found=0.0</code>), over <code>technical</code> / <code>financial</code> / <code>other</code> criteria only — eligibility criteria gate but don't score.
-----------------	---

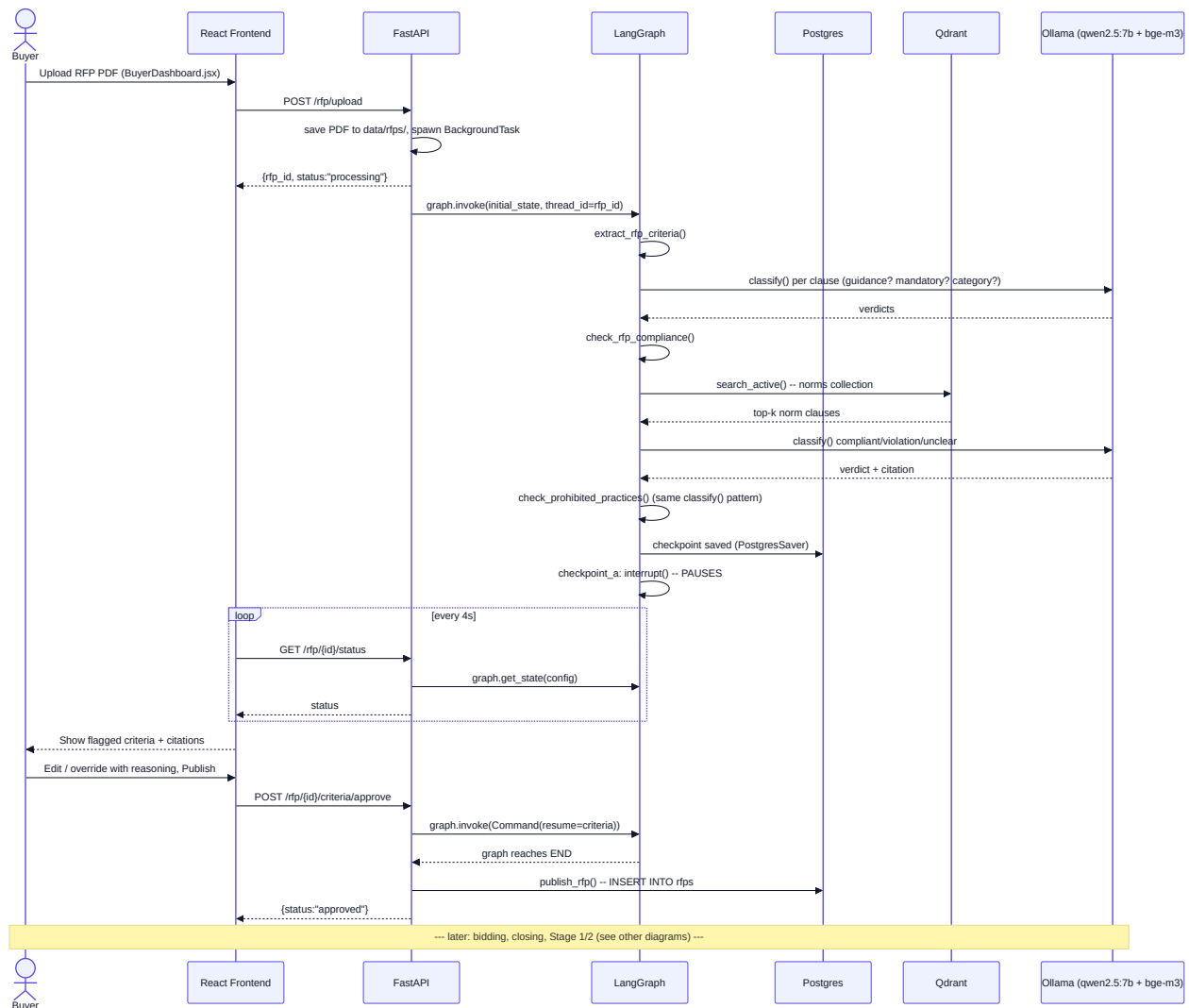
Stage 2 filtering — every criterion actually used, and where judgment sits

Step	What decides it
Price extraction	Agent: <code>extract_bid_price()</code> — LLM reads the free-form sealed Packet-II document and identifies the one "grand total" figure, distinguishing it from subtotals/per-unit rates/tax lines. This is the one criterion in Stage 2 requiring genuine judgment.
Make-in-India / local-supplier filter	Deterministic: <code>is_mii_local</code> boolean, snapshotted from the bidder's own profile declaration at submission time — every non-local bid is excluded from ranking regardless of price.
L1 ranking	Deterministic: sort remaining bids by price ascending (<code>score_stage2()</code>).
MSE price-match band	Deterministic arithmetic: is the nearest MSE bidder's price within the RFP's own stated percentage band above L1? (<code>apply_mse_price_match()</code>) — the band and quantity-split percentages are read from the specific RFP's own ATC text, never hardcoded.
QCBS blended score (if <code>evaluation_method=QCBS</code>)	Deterministic: <code>final_score = 0.7 × technical_score + 0.3 × price_score</code> (default weights; the module's own docstring flags this 70/30 split as an unvalidated placeholder — no real QCBS RFP has been seen yet to confirm GeM's typical split).
L1 price tie	Buyer-triggered: <code>run_l1_selection()</code> — a random draw among tied bidders, mirroring GeM's own documented "Run L1 Selection" feature, restricted to MSE bidders within the tie if MSE preference is active. Never automatic.

No "Top-5 shortlist" and no "Checkpoint B" exist in the current codebase. Both are named in the request that produced this document, but neither is implemented: `scoring.py` produces a *full* ranked list (`Stage2Result.ranking`), not a truncated top-5; there is no `checkpoint_b` LangGraph node, no "confirm shortlist" endpoint, and no per-bidder feedback-report generation anywhere in the code. The only two points where a human is pulled back in after Stage 1 begins are: resolving a `not_found` mandatory criterion, and breaking an L1 price tie. If your review panel expects a Checkpoint B, the honest, defensible answer is that it was part of an earlier design and is not yet built — say so rather than describing it as done.

9. End-to-End Walkthrough

A single narrated trace through the real code, naming actual functions at each hop.



Sequence diagram — RFP upload through publication. (Stage 1/2 continues this same trace; see §8's flowchart for that half.)

1. Buyer drags a PDF onto `frontend/src/pages/BuyerDashboard.jsx`, which calls `uploadRfp()` → `POST /rfp/upload`.
2. `backend/api/rfp.py` → `upload_rfp()` saves the file to `data/rfps/{rfp_id}_{filename}`, and schedules `_run_to_checkpoint_a()` as a `BackgroundTasks` job — the HTTP response returns immediately with `{rfp_id, status:"processing"}`.
3. That background job calls `graph.invoke(initial_state, config)` with `thread_id = rfp_id`. `LangGraph` runs `extract_rfp_criteria()` — chunking the PDF, then firing concurrent (max 5 at once) LLM calls to classify each candidate clause.
4. The graph advances to `check_rfp_compliance()`: for each criterion, `embed_text()` → `search_active()` against Qdrant's `norms` collection → `classify()` against Ollama.
5. Then `check_prohibited_practices()`, same `classify()` pattern, against the RFP's own extracted disclaimer list.

6. The graph reaches `checkpoint_a` and calls `interrupt()` — execution pauses, state is durably saved to Postgres via `PostgresSaver` .
7. Meanwhile, the frontend has been polling `GET /rfp/{id}/status` every 4 seconds (`BuyerDashboard.jsx` → `pollUntilReady()`) — once status flips to `awaiting_checkpoint_a` , it fetches `GET /rfp/{id}/criteria` and shows the buyer any flagged criteria inline with citations.
8. Buyer edits/overrides (with required written reasoning for anything flagged) and clicks Publish → `POST /rfp/{id}/criteria/approve` → `graph.invoke(Command(resume=edited_criteria), config)` → graph reaches `END` → `publish_rfp()` writes the real Postgres row.
9. (Later) Bidders browse `GET /bids` publicly, then a signed-in bidder submits via `POST /bidder/bids/{rfp_id}/submit` → `check_document_completeness()` hard-blocks an incomplete submission → `ingest_bid()` chunks/embeds/stores both packets in Qdrant's `bids` collection → `create_bid_record()` writes the Postgres row.
10. (Later) Closing date passes (or manual override) → `run_stage1_evaluation()` → `retrieve_and_extract_evidence()` per bid → `score_stage1()` → results saved to `bids` / `bid_evidence` tables, surfaced on `RfpManage.jsx`.
11. (Later) Buyer resolves any pending mandatory criteria, clicks "Open Financial Bids" → `run_stage2_evaluation()` → `extract_bid_price()` per qualifying bid → `score_stage2()` / `score_stage2_qcbs()` → `rfps.stage2_result` saved, status flips to `evaluated` — the final state this pipeline reaches.

10. Complete API Reference

Every endpoint, read directly from each router file. Auth column: **public** = no token required; role names indicate the FastAPI dependency gating that entire router.

/auth (public — no router-level auth dependency)

Method / Path	Purpose	Request	Response
POST /auth/login	Password login	email, password	access_token, token_type, role
POST /auth/signup/bidder	Bidder self-signup	email, password, org_name, gem_seller_proof?	access_token, token_type, role
GET /auth/me	Own profile (any role)	—	email, role, org_name, is_mse, is_mii_local, ...
PATCH /auth/me	Update org_name / seller proof	org_name, gem_seller_proof?	updated profile
POST /auth/me/change-password	Change own password	current_password, new_password	{status:"changed"}
POST /auth/me/mse-certificate	Upload MSE certificate, sets is_mse=true	file	updated profile
POST /auth/me/mii-certificate	Upload MII certificate, sets is_mii_local=true	file	updated profile

/rfp (requires buyer role)

Method / Path	Purpose	Downstream
POST /rfp/upload	Upload RFP PDF, kicks off Phase 1 in background	graph.invoke() in a BackgroundTask
GET /rfp/{id}/status	Poll current graph status	graph.get_state()
GET /rfp/{id}/criteria	Read proposed criteria at Checkpoint A	graph.get_state()
POST /rfp/{id}/criteria/approve	Publish (resumes the graph)	graph.invoke(Command(resume=...)); publish_rfp() (Postgres)
GET /rfp/mine	Buyer's own RFP list	Postgres
POST /rfp/{id}/close	Manual "Close & Evaluate Now"	close_rfp() + run_stage1_evaluation() in a BackgroundTask

GET /rfp/{id}/evaluation	Stage 1/2 results for this RFP	Postgres
POST /rfp/{id}/open-financial-bids	Trigger Stage 2	run_stage2_evaluation() in a BackgroundTask
POST /rfp/{id}/bids/{bid_id}/evidence/{criterion_id}/resolve	Human resolution of a not_found mandatory criterion	Postgres write + score_stage1() recompute
POST /rfp/{id}/run-l1-selection	Break an L1 price tie	scoring.run_l1_selection() (pure Python)

/bidder (requires bidder role)

Method / Path	Purpose	Downstream
GET /bidder/my-bids	Signed-in bidder's own submissions	Postgres
POST /bidder/bids/{rfp_id}/submit	Submit a bid (Packet-I files + Packet-II financial doc)	check_document_completeness() (hard block) → ingest_bid() (Qdrant) → create_bid_record() (Postgres)

/bids (public — no auth)

Method / Path	Purpose	Downstream
GET /bids	Search/browse published tenders	Postgres (keyword/category/status filters)
GET /bids/{id}	Tender detail	Postgres + graph.get_state()
GET /bids/{id}/document	Download the original RFP PDF	filesystem (data/rfps/)
GET /bids/{id}/legitimacy-check	Are this RFP's cited norms still active?	check_rfp_legitimacy() (deterministic keyword match) + Qdrant list_norms()
GET /bids/{id}/summary	Plain-language RFP summary	generate_rfp_summary() (LLM), cached in Postgres after first call

/admin (requires admin role)

Method / Path	Purpose	Downstream
GET /admin/norms	List all norms + status	Qdrant list_norms()
POST /admin/norms/{norm_name}/status	Change a norm's active/superseded/withdrawn status	Qdrant mark_status() (payload-only update)
GET /admin/users	List every account	Postgres
POST /admin/users/{id}/active	Suspend/restore login access	Postgres (blocks next request via is_active check)

GET /admin/flagged-rfps	Audit RFPs published despite an unresolved flag	Postgres + graph.get_state() per RFP
--------------------------------	---	--------------------------------------

11. Glossary

ATC

Additional Terms and Conditions — the free-text section of a GeM RFP where a buyer adds requirements beyond the standard template (e.g. specific technical specifications, an evaluation-method choice).

Chunking

Splitting a long document into smaller pieces small enough to embed and search individually, without losing important context at the boundaries.

Citation

A pointer back to the exact source (document, clause, page) an automated verdict was based on — so a human can verify it without re-deriving it.

Cosine similarity

A way of measuring how similar two vectors are by the angle between them, regardless of their length — the standard distance metric used by Qdrant's search in this project.

Embedding

A list of numbers that represents the meaning of a piece of text, positioned so that similar meanings end up as nearby numbers.

Edge (LangGraph)

A fixed connection from one node to the next in a graph, defining what runs after what.

GFR 2017 Rule 189

The government financial rule requiring two-envelope ("two-packet") bidding: a bidder's technical and financial documents are sealed separately, and the financial one is only opened after technical evaluation concludes.

Human-in-the-loop interrupt

A deliberate pause in an automated workflow that waits for a person to review or decide something before continuing — implemented here via LangGraph's `interrupt()`.

JWT

JSON Web Token — a signed, tamper-proof token issued at login that proves who a user is (and their role) on every subsequent request, without the server needing to look up a session on every call.

LangGraph node

One step in a LangGraph workflow — a function that receives the current state and returns updates to it.

MII (Make in India)

DPIIT's policy giving purchase preference to "local" suppliers (Class-I/Class-II, by minimum local-content percentage) over non-local ones in government procurement.

MSE / MSME

Micro and Small Enterprise — a business size category that receives mandated procurement preference under the 2012 Policy Order (e.g. a price-match opportunity within a band above the lowest bid).

Packet-I / Packet-II

The two sealed parts of a bid — Packet-I (technical/eligibility documents) and Packet-II (the financial/price document), kept isolated per GFR Rule 189.

QCBS

Quality- and Cost-Based Selection — an evaluation method that blends a bid's technical quality score with its price into one combined ranking, as opposed to price-only (L1).

RAG (Retrieval-Augmented Generation)

Answering a question by first retrieving relevant source text from an external store, then handing that text to a language model, rather than relying on facts baked into the model's own training.

RRF (Reciprocal Rank Fusion)

A technique for combining rankings from multiple search methods into one score. Not used anywhere in this codebase — this project runs a single dense-vector search per query, not a fusion of multiple retrieval methods.

State (LangGraph)

The shared data object passed between nodes in a graph, which each node reads from and returns updates to.

Vector DB

A database designed to store embeddings and quickly find the ones most similar to a given query — Qdrant, in this project.

12. Anticipated Review Questions & Answers

1. Why a local model (Ollama) instead of GPT-4/Claude/Gemini?

Data residency — tender and bid documents shouldn't leave infrastructure the buyer controls — plus zero per-token cost for a system meant to run many classification calls per document.

2. How do you guarantee auditability?

Every automated verdict either carries a citation (source document, clause, page) or is downgraded to `not_found` rather than trusted uncited. The final scoring arithmetic is deterministic, pure Python with no LLM involved, so the same evidence always produces the same result — reproducible on demand, not just "the model said so."

3. Why two databases (Postgres + Qdrant) instead of one?

They answer fundamentally different questions — Postgres answers structured, relational questions (which bids belong to this RFP); Qdrant answers semantic-similarity questions (which norm clauses are conceptually related to this criterion). Forcing one engine to do both well is the harder path, not the simpler one.

4. What happens if a norm is superseded mid-evaluation?

Nothing retroactive happens to evaluations already run against the old version — `mark_status()` only affects future `search_active()` calls. The old chunks aren't deleted, just excluded from future active-norm searches, so a past evaluation's citations remain traceable to the version that was active when it ran.

5. Why deterministic scoring instead of letting the LLM produce a score directly?

Auditability, not a capability gap. See §2's callout — an explainable formula beats "the model said so," independent of how good the model is. (Though it's also true the original small model was empirically unreliable at numeric comparisons — a second, reinforcing reason.)

6. Is this actually multi-agent, or just several LLM calls wrapped in functions?

Both are true, and that's fine — "agent" here means a step that requires genuine judgment (see §4's table) rather than a lookup or regex. What makes the system as a whole agentic rather than a script is LangGraph's durable, resumable orchestration around those calls, not the individual calls being exotic.

7. Why does Checkpoint A require written reasoning to override a flag?

So a flagged-but-published tender has a permanent, attributable record of why a human disagreed with the automated flag — auditable after the fact, not just a silent click-through.

8. What stops a bidder's financial bid from being read early?

`search_bid()` defaults its packet parameter to Packet-I; the only function that ever reads Packet-II (`extract_bid_price()`) is called exclusively from Stage 2, which only runs after a buyer's explicit action and only for Stage-1-passed bids. It's enforced in code structure, not just workflow convention.

9. How do you handle an RFP with no clear Evaluation Method stated?

Defaults to L1 — the only value actually observed in real documents so far, per `_extract_evaluation_method()`'s own comment.

10. What happens to a bid missing a required document?

Blocked at submission time (hard 422 error, not a warning) — deliberately stricter than real GeM, which the code comments note "silently invalidates a bid with a missing document with no warning at all."

11. Why is a mandatory criterion's "not_found" verdict not treated as a fail?

Absence of matching content could mean genuine non-compliance or could mean a retrieval miss — those are different failure modes, and only a human should decide which. It blocks progress to Stage 2 rather than silently passing or failing.

12. Is there a mechanism to resume an evaluation that crashed halfway?

Between LangGraph nodes, yes — Postgres-backed checkpointing survives a full process restart. Mid-node, no — if a node's internal loop dies partway (e.g. a network failure to the LLM server), that node must re-run from its own start; there's no finer-grained resume. The API layer now writes an explicit `status="failed"` so this is visible rather than a silent hang.

13. Why is chunking section-aware instead of a fixed-size sliding window?

A chunk blending the tail of one clause with the head of an unrelated one produces confidently wrong retrieval, not just a messy chunk — two unrelated rules can discuss the same general topic (e.g. a discount percentage) under completely different conditions.

14. How is the QCBS 70/30 technical/price weighting justified?

It isn't yet, and the code says so directly — it's an explicit placeholder default, since no real QCBS-evaluated RFP has been seen to validate against. This should be stated plainly if asked, not defended as a confirmed constant.

15. Why is a superseded norm never deleted?

Non-destructive by design — a past evaluation must remain traceable to exactly the norm version that was active when it ran; deleting old versions would break that audit trail.

16. How does deactivating a user account actually take effect?

Immediately, not at next login — every authenticated request re-checks `is_active` against Postgres (see `backend/auth.py` → `_check_active()`), closing the gap where an already-issued JWT would otherwise stay valid until its 24-hour expiry.

17. What's the actual difference between the RFP-compliance check and the prohibited-practices check?

Compliance checks a criterion against the external government-norms knowledge base; the prohibited-practices check is narrower and entirely self-contained — it checks a criterion against *that same RFP's own* disclaimer page listing GeM's known buyer drafting-mistakes, with no Qdrant retrieval involved (the list is short enough to put directly in the prompt).

18. Does the system ever auto-select a winner?

No. Stage 2 produces a fully ranked list and a computed L1/QCBS winner field, but nothing in the code marks a bid as "awarded" or triggers any downstream contractual action — the ranked result is the system's final output, for a human to act on.

Appendix — Codebase Inventory

Directory tree (source only — venv/, node_modules/, __pycache__/, .git/ omitted)

```
backend/
├─ api/          admin.py, auth.py, bidder.py, bids.py, rfp.py
├─ graph/        build_graph.py, state.py,
│                 extract_rfp_criteria.py, check_rfp_compliance.py,
│                 check_prohibited_practices.py, check_document_completeness.py,
│                 check_rfp_legitimacy.py, generate_rfp_summary.py,
│                 extract_bid_price.py, retrieve_and_extract_evidence.py,
│                 run_stage1_evaluation.py, run_stage2_evaluation.py
├─ llm/          ollama_client.py
├─ models/       rfp.py, evidence.py
├─ rag/          embeddings.py, qdrant_client.py
├─ scoring/      scoring.py
├─ auth.py, db.py, logging_config.py, main.py

ingestion/       extract_text.py, extract_tables.py, language_filter.py,
                  chunker.py, ingest_norms.py, ingest_bid.py

scripts/         verify_env.py, verify_infra.py, query_norms.py

frontend/src/
├─ pages/        Landing, About, BuyerLogin/Dashboard/Rfps, RfpManage,
│                 BidderLogin/Signup/Dashboard, BidSubmission,
│                 AdminLogin/Dashboard, Bids, BidDetail, Profile
├─ components/   Nav, Container, Footer, EvaluationResult, RfpUploadForm,
│                 KpiStrip, ProtectedRoute, AccountMenu, LoginMenu, ThemeToggle
├─ context/      AuthContext.jsx, ThemeContext.jsx
├─ api/          client.js

data/norms/      6 ingested source PDFs + manifest.json (+ 1 not-yet-ingested scan)
data/bids/       per-bid_id subfolders, both packets
data/certificates/ uploaded MSE/MII certificates

tests/          test_scoring.py
docker-compose.yml Qdrant + Postgres
requirements.txt  Python dependencies
```

Qdrant collections

Collection	Vector size	Key payload fields	Status filter?
<code>norms</code>	1024 (bge-m3)	doc_id, norm_name, status, version, clause_ref, chunk_type, page_number, text	Yes — active/superseded/withdrawn
<code>bids</code>	1024 (bge-m3)	bid_id, rfp_id, bidder_name, source_file, packet (I/II), chunk_type, page_number, text	No status filter (isolation is by bid_id + packet instead)

Things this document could NOT confirm from the code, or that are explicitly not implemented

- `STATUS_REPORT.md` is referenced by name in code comments (e.g. `extract_rfp_criteria.py`) but does not exist anywhere in the current repository.
- A "buyer-side, non-blocking" use of `check_document_completeness()` is described in that module's own docstring, but the only place it's actually wired into the API is the bidder-submission hard block (`backend/api/bidder.py`) — no buyer-facing completeness review screen exists in the current frontend/API.
- No LlamaIndex usage anywhere (see §2).
- No Checkpoint B, no Top-5 shortlist, no per-bidder feedback report (see §8).
- No separate "Competent Authority"/"Bid Evaluation Committee" role — three roles exist: buyer, bidder, admin.
- `threshold_check.py` (a deterministic numeric-threshold checker) existed historically and was deleted once a larger LLM proved reliable — only a stale compiled bytecode cache of it remains on disk, not the source.